AI-Powered Resource Allocation & Bench Management with RAG

CAPSTONE PROJECT

AI Training for Engineers

Prepared by

AI Academy & Learning and Development Team

Table of Contents

AI-Powered Resource Allocation & Bench Management with RAG

1. Problem Statement

IT services organizations often struggle to allocate the right employees to the right projects due to:

- Limited visibility into employee skills and availability
- High bench costs from underutilized resources
- Manual and time-consuming staffing processes
- Difficulty identifying suitable resources quickly
- Lack of insight into future skill demand

These challenges impact resource utilization, project delivery timelines, employee engagement, and profitability.

2. Proposed Solution

Build an AI-powered Resource Allocation & Bench Management platform using Retrieval-Augmented Generation (RAG) to help resource managers identify the best-fit employees for project opportunities.

The solution analyzes employee skills, experience, certifications, project history, utilization, and availability to generate explainable staffing recommendations.

The platform provides two primary user experiences:

AI Assistant

A conversational chat interface enables resource managers to submit staffing requests using natural language and receive AI-generated recommendations with match scores and justifications.

Analytics Dashboard

An interactive dashboard provides real-time visibility into workforce utilization, bench status, skill demand, resource availability, and future staffing needs.

Together, these capabilities enable faster staffing decisions, improved resource utilization, and proactive workforce planning.

3. Business Impact

- Reduce bench costs through proactive resource allocation
- Improve overall employee utilization
- Accelerate project staffing decisions
- Increase project delivery efficiency
- Identify skill gaps and upskilling opportunities
- Enable data-driven workforce planning using bench and utilization analytics
- Improve visibility into resource availability and future demand

4. How RAG Works

The system retrieves relevant information from:

- Employee resumes
- Skill matrices
- Certifications
- Project history
- Bench profiles
- Project requirement documents

When a resource manager submits a staffing request, the RAG engine retrieves the most relevant employee information, and the LLM generates recommendations with match scores and explanations.

Example Request

Find three available Python developers with Azure experience for a healthcare project starting next month.

Example Output

- Employee A – 92% match
- Employee B – 87% match
- Employee C – 85% match

5. Key Features

- AI-powered resource matching
- Conversational staffing assistant
- Bench management
- Skill gap identification
- Upskilling recommendations
- Resource utilization dashboard
- Bench analytics
- Demand forecasting
- Explainable AI recommendations

6. High-Level Architecture

6.1 AI Assistant Architecture (RAG-Based Resource Recommendation)

```
text id="z9l1kg" +-----------------------+ |  Resource Manager    | +---------
-+-----------+            |            v +-----------------------+ | Chat
Interface       | | (Natural Language)   | +----------+-----------+
|            v +-----------------------+ | Prompt Processing &  | | Metadata
Filters     | +----------+-----------+            |            v +-----------
----------+ | Embedding Generation | +----------+-----------+            |
v +-----------------------+ | Vector Database      | | (FAISS)            |
+----------+-----------+            |            v +----------------------+ |
```

```
RAG Retrieval Engine | +----------+----------+            |            v +--
--------------------+ | LLM Recommendation    | | Engine               | +----
------+----------+            |            v +----------------------+ |
Match Scores,        | | Explanations &       | | Upskilling Insights  | +---
-------------------+
```

Data Sources:

- Employee resumes
- Skill matrices
- Certifications
- Project history
- Availability status
- Bench profiles
- Project requirement documents

6.2 Analytics Dashboard Architecture (SQL-Based Analytics)

```
text id="m5x2hf" +----------------------+ |  Resource Manager    | +---------
-+-----------+            |            v +----------------------+ | Analytics
Dashboard  | +----------+----------+            |            v +------------
----------+ | Dashboard API Layer  | | (FastAPI)            | +----------+---
--------+            |            v +----------------------+ | SQL Queries &
| | Business Rules       | +----------+----------+            |            v
+----------------------+ | PostgreSQL / MySQL   | +----------+----------+
|            v +----------------------+ | Employee, Project &  | | Allocation
Data       | +----------------------+
```

Dashboard Metrics:

- Total employees
- Available resources
- Bench percentage
- Resource utilization
- Bench by skill
- Skill demand trends
- Upcoming project demand
- Historical allocation trends

7. Technology Stack

Component	Technology
Frontend	React
Backend	FastAPI
AI Framework	LangChain
Embeddings	OpenAI Embeddings
Vector Database	FAISS
LLM	Open-source LLM / OpenAI
Database	PostgreSQL / MySQL

8. MVP Dataset

Employee Data

- Employee ID
- Skills
- Experience
- Certifications
- Availability
- Current utilization
- Resume

Project Data

- Required skills
- Domain
- Start date
- Duration

Historical Allocation Data

- Employee-project mapping
- Performance ratings

9. Implementation Steps

Phase 1: Data Preparation

- Collect employee and project data
- Define a standardized skill taxonomy
- Store data in CSV or JSON format

Phase 2: Build the RAG Pipeline

- Ingest employee and project documents
- Generate embeddings
- Store embeddings in FAISS

Phase 3: Retrieval

- Convert staffing requests into embeddings
- Retrieve top-matching employees
- Filter results by availability and utilization

Phase 4: Recommendation Engine

Generate:

- Recommended employees
- Match scores
- Recommendation justifications
- Skill gaps
- Upskilling suggestions

Phase 5: Dashboard Development

Display:

- Available resources
- Bench percentage
- Utilization trends
- Skill demand insights
- Staffing recommendations

10. User Experience

Dashboard

- Resource utilization metrics
- Bench analytics
- Available employees
- Skill demand trends
- Upcoming project requirements

AI Assistant

Managers can submit staffing requests using natural language.

Example:

> "Suggest available React developers with 5+ years of experience."

The system returns recommended employees, match scores, and reasons for selection.

11. Sample Chat Interface

```text id="m7z4kq" +——————————————————————+ | Resource Allocation Assistant | +——————————————————————+

Manager: Find two available Java developers with AWS experience.

AI Response:

✓ Employee A – 91% Match ✓ Employee B – 88% Match

Reasoning: • Skills match project requirements • Availability confirmed • Relevant project experience

[View Profile] [Allocate Resource]

```
---

12. Sample Dashboard
```
```

````
```text id="d3w8pv"
+--------------------------------------------------------+
| AI Resource Allocation Dashboard                       |
+--------------------------------------------------------+

+-------------+-------------+-------------+-------------+
| Employees   | Available   | Bench %     | Utilization |
| 250         | 42          | 16.8%       | 78%         |
+-------------+-------------+-------------+-------------+

+-----------------------+-------------------------------+
| Skill Demand          | Bench by Skill                |
| Python ████████       | Java ████                     |
| Azure  ██████         | React ███                     |
| GenAI  █████          | Testing ██                    |
+-----------------------+-------------------------------+

+--------------------------------------------------------+
| Upcoming Projects                                      |
| Healthcare AI – 5 resources needed                     |
| Banking Portal – 3 resources needed                    |
+--------------------------------------------------------+
````

====================== *End of the document* =========================
````